CampusIQ – Day 1 Notes

Name: Gufran Date: 18 October 2026 Role: AI (/ai folder)

Topic: Laptop setup, Git, GitHub, first Python

0. The project in one minute

- CampusIQ = an AI assistant for our college. It answers questions from regulation PDFs, marks and attendance.
- Built by 2 people: **Gufran** = AI brain (`/ai`), **Fahad** = data + app (`/backend`, `/frontend`).
- Planned pieces: RAG (search + answer), Text-to-SQL, LangGraph agents, evaluation, FastAPI, Streamlit, Docker.
- Repo: `github.com/gufran-fahad-campusiq/campusiq` (private, inside a GitHub organization).

1. Key words

- Git = tool on my laptop that saves code and its full history (version control).
- GitHub = website that stores Git projects online (like Google Drive for code).
- Repository (repo) = the project folder that Git tracks.
- Clone = copy a repo from GitHub to my laptop. Done only once.
- Commit = a save point with a short message.
- Push = upload my commits to GitHub. Pull = download new commits from GitHub.
- Organization = a shared GitHub account, both of us are **Owners**.
- .gitignore = list of files Git must NOT upload: .env, API Keys, big data.

2. Tools I installed

Tool	Why	Check in terminal
Python 3.10.11	main language	`python --version`
Git 2.53	save and share code	`git --version`
VS Code	editor + terminal	opens, runs files

While installing Python, tick "Add Python to PATH". If not ticked, the command `python` will not be found.

3. Tell Git who I am (one time only)

```
git config --global user.name  "Gufran"
git config --global user.email "gufrananas23@gmail.com"
git config --global --list        # check both values
```

- The **name** is just a display name on commits.
- The **email** is what links commits to my GitHub profile.

4. Clone the repo

```
cd %USERPROFILE%\Documents
git clone https://github.com/gufran-fahad-campusiq/campusiq.git
cd campusiq
```

- Private repo: the first time, a browser window opens. Click Authorize.
- Always check the prompt ends with `...\Documents\campusiq>` before running git commands.
- Mistake I saw: `git clone git clone URL`. Keep only one `git clone`.

5. Folder layout of the repo

```
campusiq/
  ai/              my code (Gufran)
  backend/         Fahad's code
  notes/
    gufran/        my daily notes (PDF)
    fahad/         Fahad's daily notes
  README.md        front page of the repo
  .gitignore       files Git ignores
```

6. Daily Git cycle (learn by heart)

1. `git pull` - morning: get my friend's latest work.
2. `git status` - what changed? (red = not staged, green = staged)
3. `git add .` - stage all changed files.
4. `git commit -m "day 1: message"` - make a save point.
5. `git push` - upload to GitHub.

- Good commit message: short and clear, like `"day 1: python basics"`.
- To undo a pushed mistake use `git revert`. Never use `--force` or `reset --hard`.

Never push passwords, API keys or .env files. If it happens, revoke the key immediately.

7. Python basics

- **Variable** = a label on a box that stores a value.
- Types: *str* "text", *int* 5, *float* 8.4, *bool* True or False.
- `print(...)` shows output. `#` starts a comment.
- *f-string*: `f"{name} has {cgpa}"` puts variables inside text.

```
name = "Gufran"
cgpa = 8.4
print(f"{name} has CGPA {cgpa}")      # Gufran has CGPA 8.4
```

List vs dictionary

```
marks = [78, 85, 90]          # list: items in order, index from 0
marks[0]            -> 78
marks[-1]           -> 90              # last item
marks.append(66)                       # add item, len(marks) = 4

st = {"name": "Gufran", "cgpa": 8.4}   # dict: key -> value
st["name"]          -> "Gufran"
st["branch"] = "AIML"                  # add a new key
```

- *list = look up by **number** (position). dict = look up by **key** (name).*

if, for, def

```
if cgpa > 8:                    # decision
    print("good")
for m in marks:                 # repeat for every item
    print(m)
def pct(attended, total):       # reusable function
    return attended / total * 100
print(round(pct(41, 60), 1))    # 68.3
```

- *A line ending with `:` opens a block. The next lines are indented 4 spaces.*
- `def` *= write a recipe once, use it many times.* `return` *sends the answer back.*

8. My mistake: indentation

```
for s in students:
    if s["cgpa"] > 8:
        print(s["name"], "above 8")
        if s["cgpa"] < 7.5:          # WRONG: inside the first if
            print(s["name"], "below 7.5")
```

- Indentation tells Python which code belongs inside which block.
- The second `if` was inside the first, so it ran only for CGPA above 8 - it could never be true.
- Fix: put both `if` lines at the same level, both inside the `for`.

```
for s in students:
    if s["cgpa"] > 8:
        print(s["name"], "has CGPA above 8")
    if s["cgpa"] < 7.5:
        print(s["name"], "has CGPA below 7.5")
```

9. Common errors and fixes

Error	Cause and fix
`not a git repository`	wrong folder. Go into `campusiq` (check the prompt path).
`cd campusiq` not found	I was already inside it. Do not cd again.
push goes to old URL	repo moved to the organization: `git remote set-url origin NEW_URL`.
friend cannot see repo	invitation not accepted. Accept at `github.com/.../invitations`.

10. Self-check (answers)

- Clone vs commit vs push? Clone = copy repo once, commit = local save point, push = upload.
- Why set the email in Git? It links my commits to my GitHub profile.
- First list index? 0. Read a dict value? `d["key"]`.
- What does `.gitignore` do? Keeps secrets and big files out of GitHub.

11. Quick revision sheet

```
GIT   (every day)
  git pull                              # morning
  git status                            # what changed?
  git add .                             # stage everything
  git commit -m "day N: what I did" # save point
  git push                              # upload

PYTHON
  x = 5   name = "A"   ok = True    # variables
  marks = [1, 2, 3]   marks[0]      # list by index
  st = {"k": 1}       st["k"]       # dict by key
  if a > b:   for i in items:   def f(x): return x
  # after ":" indent the next line by 4 spaces
```

Day 1 complete. Next: Day 2 - virtual environment (venv), pip, reading text files and splitting text into chunks.